HW3 Instructions

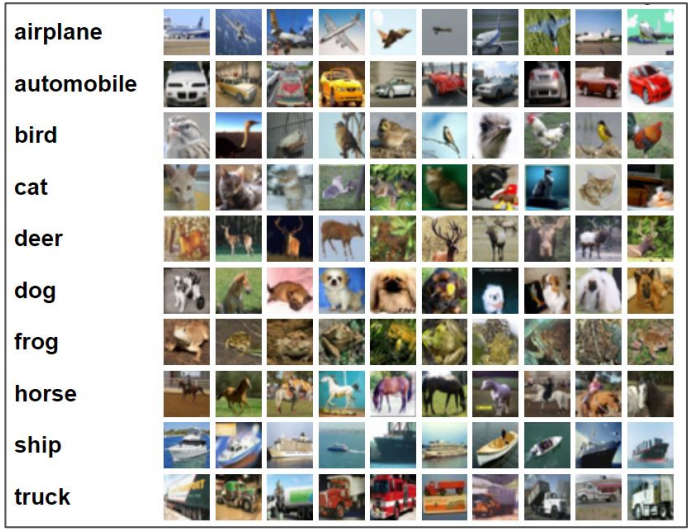
2026/05/12 Tues. 12:00 ~ 2026/06/01 Mon. 11:59

Tasks

- Learning to design a Transformer architecture.
- Learning to implement single (and multi-head) attention mechanism.
- Learning to implement the position-wise feedforward (FFN) Layer.

Dataset: CIFAR-10

- 32 x 32 color images
- 10 classes, with 6000 images per class
- Training set: 50,000 images
- Test set: 10,000 images
- The classes are completely mutually exclusive



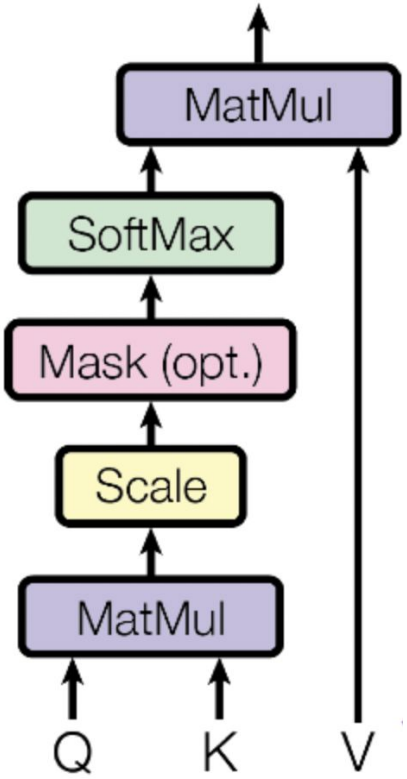
2026_DL_HW3.ipynb: Code Overview

- Part 1: Import Libraries
- Part 2: Global Variables
- Part 3: Data Preprocessing & Data Loading
- Part 4: Image Tokenization
- Part 5: Position-wise Feedforward Layer **(HW3 Implement code)**
- Part 6: Attention Mechanism **(HW3 Implement code, Bonus)**
- Part 7: Transformer Architecture **(HW3 Implement code)**
- Part 8: Vision Transformer Construction
- Part 9: Model Hyperparameters **(HW3 Adjustable)**
- Part 10: Loss Function
- Part 11: Optimizer
- Part 12: Training
- Part 13: Testing & Submission Generation

2026_DL_HW3.ipynb: Part5 Position-wise Feedforward Layer

$$FFN(x) = max(0, xW_1 + b_1)W_2 + b_2$$

2026_DL_HW3.ipynb: Part6 Attention Mechanism



Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

$$\begin{aligned} \alpha_{ij} &= \text{softmax}(score(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) \;\; \forall j \in e) \\ &= \frac{\exp(score(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e)}{\sum_k \exp(score(\mathbf{h}_{i-1}^d, \mathbf{h}_k^e))} \end{aligned}$$

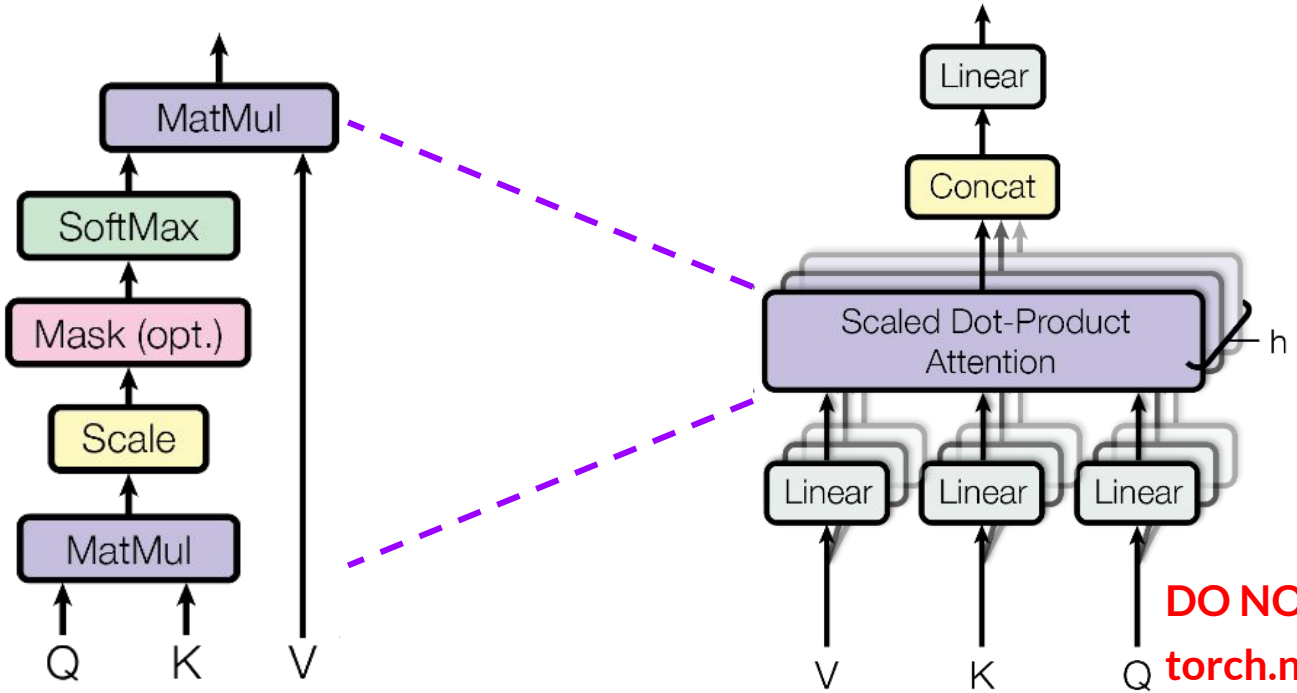
$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e$$

DO NOT directly use torch.nn.MultiheadAttention()

2026_DL_HW3.ipynb: Part6 Attention Mechanism

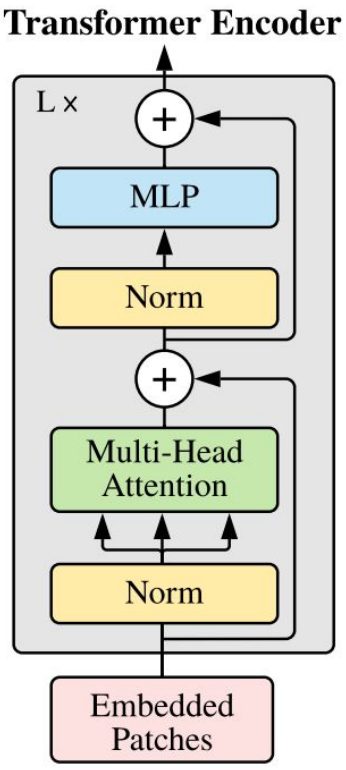
Scaled Dot-Product Attention

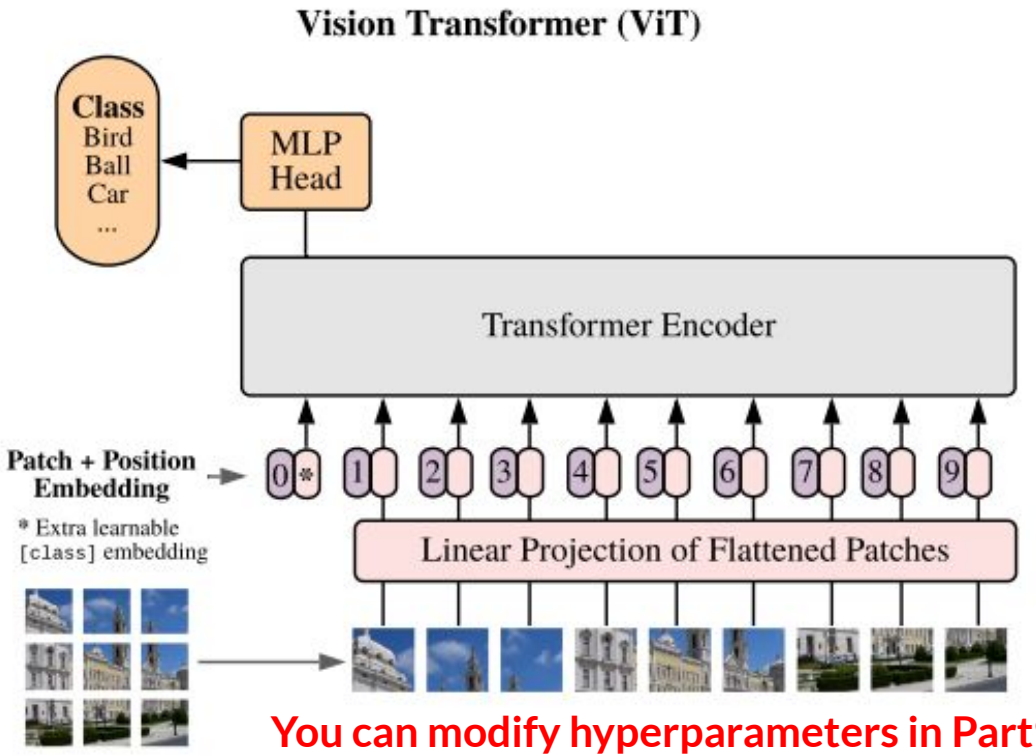
Multi-head Attention (Bonus 20% credits)



DO NOT directly use torch.nn.MultiheadAttention()

2026_DL_HW3.ipynb: Part7 Transformer Architecture





You can modify hyperparameters in Part9

Grading (1/2)

- Part 5: Position-wise feedforward (FFN) Layer **20%**
- Part 6: Single-head attention **20%**
- Part 6: Multi-head attention **20% (Bonus)**
- Part 7: Transformer architecture **20%**

Grading (2/2)

- Model size **15%**: (size of .pt file)
  - 5%: If your model is smaller than **1MB**, you will get 5%.
  - 10%: The remaining 10% will depend on your ranking within the class.
- Model accuracy **15%**:
  - 5%: If your accuracy is higher than **57%**, you will get 5%.
  - 10%: The remaining 10% will depend on your ranking within the class.
- Model accuracy on another dataset **10%**: it will depend on your ranking within the class.

Rules

- You can use all torch and einops functions for your implementations
- You can modify hyperparameters in part 9
- **DO NOT** use **torch.nn.MultiheadAttention()**
- **DO NOT** attempt to modify the sections **DO NOT MODIFY**.

Submission

- Upload your csv file to Kaggle Competition
- **Remember to change your Kaggle Competition team name into StudentID**
- Upload your zip file to NTU Cool
- StudentID_HW3.zip
  - DL_HW3_StudentID.ipynb
  - StudentID_submission.pt
  - StudentID_submission.csv
- Deadline: **2026/06/01 (Mon.) 11:59**

TA Hour

- Time: By appointment
- Location: 教研館 317
- If you have any question, please send email to TA first.
- TA's email: r14725041@ntu.edu.tw & r14725042@ntu.edu.tw
- CC both TAs in your emails.

Supplemental Materials
Einops

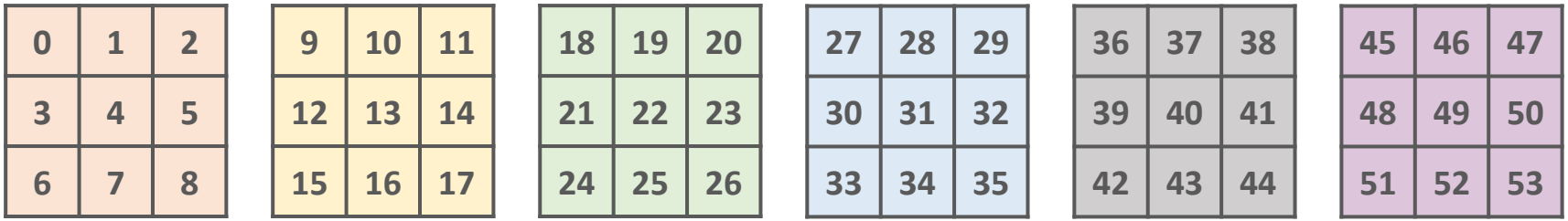
2026/05/12 Tues.

What is Einops?

- Module for flexible and powerful tensor operations
- Make code more readable and reliable
- Support numpy, pytorch, tensorflow and other modules
- Installation: **pip install einops**
- Documentation: https://einops.rocks

Einops: Exmaple

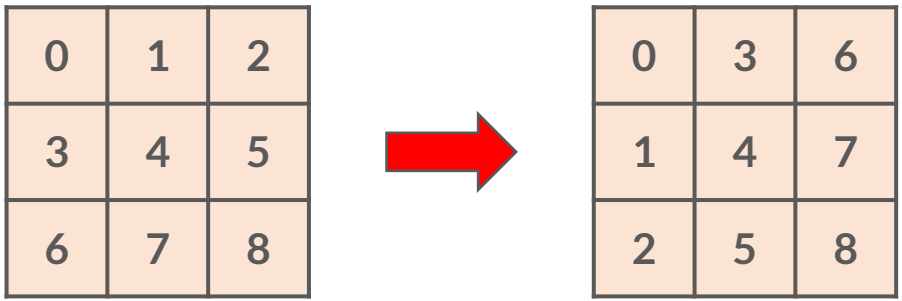
- Shape: E(6, 3, 3)
- batch = 6, height = 3, width = 3



Einops: Rearrange (1/12)

- Compose batch and height to a new dimension
- Transposition
- **from einops import rearrange**

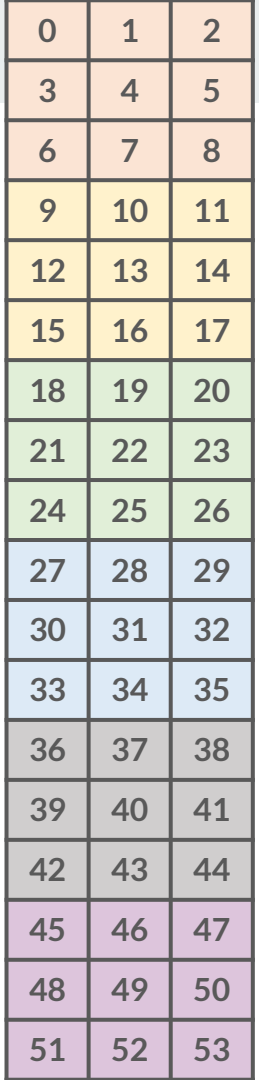
  rearrange(E[0], ‘h w -> w h’)



Einops: Rearrange (2/12)

- Concatenate
- **rearrange(E, ‘b h w -> (b h) w’)**
- batch = 6, height = 3, width = 3

  Original shape: (6, 3, 3) -> New shape: (18, 3)



Einops: Rearrange (3/12)

- Concatenate
- **rearrange(E, ‘b h w -> h (b w)’)**
- batch = 6, height = 3, width = 3

  Original shape: (6, 3, 3) -> New shape: (3, 18)

0	1	2	9	10	11	18	19	20	27	28	29	36	37	38	45	46	47
3	4	5	12	13	14	21	22	23	30	31	32	39	40	41	48	49	50
6	7	8	15	16	17	24	25	26	33	34	35	42	43	44	51	52	53

Einops: Rearrange (4/12)

- Different sort
- **rearrange(E, ‘b h w -> h (w b)’)**
- batch = 6, height = 3, width = 3

  Original shape: (6, 3, 3) -> New shape: (3, 18)

0	9	18	27	36	45	1	10	19	28	37	46	2	11	20	29	38	47
3	12	21	30	39	48	4	13	22	31	40	49	5	14	23	32	41	50
6	15	24	33	42	51	7	16	25	34	43	52	8	17	26	35	44	53

Einops: Rearrange (5/12)

- Flatten
- **rearrange(E, ‘b h w -> (b h w)’)**
- batch = 6, height = 3, width = 3

  Original shape: (6, 3, 3) -> New shape: (54,)

Einops: Rearrange (6/12)

- Decomposition of axis
- **rearrange(E, '(b1 b2) h w -> (b1 h) (b2 w)', b1=2)**
- batch = 6, height = 3, width = 3

  Original shape: (6, 3, 3) -> New shape: (6, 9)

0	1	2	9	10	11	18	19	20
3	4	5	12	13	14	21	22	23
6	7	8	15	16	17	24	25	26
27	28	29	36	37	38	45	46	47
30	31	32	39	40	41	48	49	50
33	34	35	42	43	44	51	52	53

Einops: Rearrange (7/12)

- Decomposition of axis
- **rearrange(E, '(b1 b2) h w -> (b2 h) (b1 w)', b1 = 2)**
- batch = 6, height = 3, width = 3

  Original shape: (6, 3, 3) -> New shape: (9, 6)

0	1	2	27	28	29
3	4	5	30	31	32
6	7	8	33	34	35
9	10	11	36	37	38
12	13	14	39	40	41
15	16	17	42	43	44
18	19	20	45	46	47
21	22	23	48	49	50
24	25	26	51	52	53

Einops: Rearrange (8/12)

- Move part of width dimension to height
- **rearrange(E, 'b h (w w2) -> (h w2) (b w)', w2 = 3)**
- batch = 6, height = 3, width = 3

  Original shape: (6, 3, 3) -> New shape: (9, 6)

0	9	18	27	36	45
1	10	19	28	37	46
2	11	20	29	38	47
3	12	21	30	39	48
4	13	22	31	40	49
5	14	23	32	41	50
6	15	24	33	42	51
7	16	25	34	43	52
8	17	26	35	44	53

Einops: Rearrange (9/12)

- Move part of height dimension to width
- **rearrange(E, ‘b (h h2) w -> (b h) (h2 w)’, h2 = 3)**
- batch = 6, height = 3, width = 3

  Original shape: (6, 3, 3) -> New shape: (6, 9)

0	1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26
27	28	29	30	31	32	33	34	35
36	37	38	39	40	41	42	43	44
45	46	47	48	49	50	51	52	53

Einops: Rearrange (10/12)

- Reorder
- **rearrange(E, '(b1 b2) h w -> h (b2 b1 w)', b1 = 2)**
- batch = 6, height = 3, width = 3

  Original shape: (6, 3, 3) -> New shape: (3, 18)

0	1	2	27	28	29	9	10	11	36	37	38	18	19	20	45	46	47
3	4	5	30	31	32	12	13	14	39	40	41	21	22	23	48	49	50
6	7	8	33	34	35	15	16	17	42	43	44	24	25	26	51	52	53

Einops: Rearrange (11/12)

- Expand dimensions
- **rearrange(torch.arange(3), 'd -> d 1')**

  tensor([0, 1, 2]) -> tensor([[0], [1], [2]])
- Squeeze
- **rearrange(last_tensor, 'd 1 -> d')**

  tensor([[0], [1], [2]]) -> tensor([0, 1, 2])

Einops: Rearrange (12/12)

- from einops.layers.torch import Rearrange

```
class myTokenization(nn.Module):

    def __init__(self, output_dim, patch_size, channels):

        super().__init__()

        patch_dim = patch_size * patch_size * channels

        self.to_patch_tokens = nn.Sequential(
            Rearrange('b c (h p1) (w p2) -> b (h w) (p1 p2 c)', p1 = patch_size, p2 = patch_size),
            nn.LayerNorm(patch_dim),
            nn.Linear(patch_dim, output_dim)
        )

    def forward(self, x):

        return self.to_patch_tokens(x)
```

Einops: Reduce (1/5)

- Calculate mean over batch
- **from einops import reduce**

  reduce(E.float(), 'b h w -> h w', 'mean')
- batch = 6, height = 3, width = 3

  Original shape: (6, 3, 3) -> New shape: (3, 3)

22.5	23.5	24.5
25.5	26.5	27.5
28.5	29.5	30.5

Einops: Reduce (2/5)

- Sum over batch
- **reduce(E, 'b h w -> h w', ‘sum’)**
- batch = 6, height = 3, width = 3

  Original shape: (6, 3, 3) -> New shape: (3, 3)

135	141	147
153	159	165
171	177	183

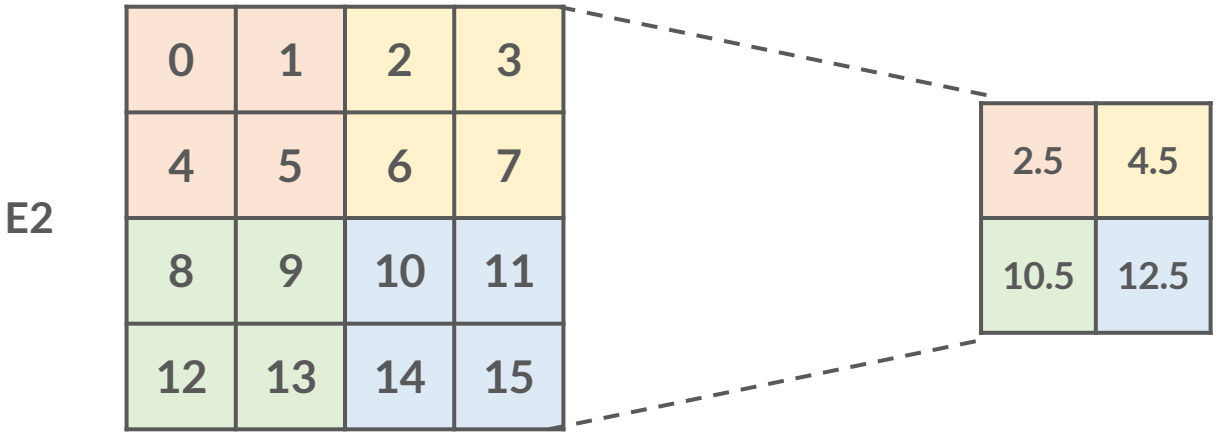
Einops: Reduce (3/5)

- Compute max in each image individually
- **reduce(E, 'b h w -> b () ()', 'max')**
- batch = 6, height = 3, width = 3

  Original shape: (6, 3, 3) -> New shape: (6, 1, 1)

Einops: Reduce (4/5)

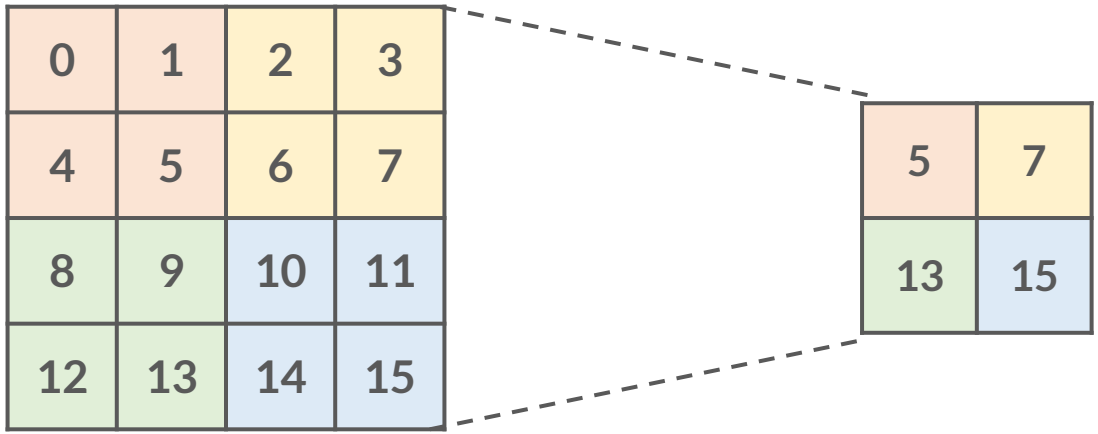
- Average pooling
- **reduce(E2.float(), '(h h2) (w w2) -> h w', 'mean', h2 = 2, w2 = 2)**
- equals to **avg_pool2d(kernel_size=2, stride=2)**



Einops: Reduce (5/5)

- Max pooling
- **reduce(E2, '(h h2) (w w2) -> h w', 'max', h2 = 2, w2 = 2)**
- equals to **max_pool2d(kernel_size=2, stride=2)**

E2

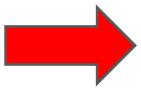


Einops: Repeat (1/3)

- Repeat along more existing axes
- **from einops import repeat**

  repeat(E[0], 'h w -> h (repeat w)', repeat = 3)

0	1	2
3	4	5
6	7	8

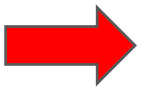


0	1	2	0	1	2	0	1	2
3	4	5	3	4	5	3	4	5
6	7	8	6	7	8	6	7	8

Einops: Repeat (2/3)

- Repeat along more existing axes
- repeat(E[0], 'h w -> (2 h) (2 w)')

0	1	2
3	4	5
6	7	8

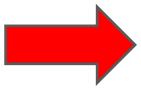


0	1	2	0	1	2
3	4	5	3	4	5
6	7	8	6	7	8
0	1	2	0	1	2
3	4	5	3	4	5
6	7	8	6	7	8

Einops: Repeat (3/3)

- Repeat each element
- repeat(E[0], 'h w -> h (w 2)')

0	1	2
3	4	5
6	7	8



0	0	1	1	2	2
3	3	4	4	5	5
6	6	7	7	8	8